

Procedure

1. Open a code editor and create a new C source file that defines a function containing a fixed size character buffer of 16 bytes.
2. Inside that function, copy a user-supplied command line argument into the buffer using the standard library function `strcpy()`, without performing any length check on the input, and save the file.
3. Compile the program using `gcc` and first execute it with a short input string to verify that it behaves correctly and prints the buffer content as expected.
4. Execute the same compiled program again, this time supplying an input string considerably longer than the 16 byte buffer, and observe that the program terminates abnormally with a segmentation fault because the oversized input has overwritten adjacent memory on the stack.
5. Create a second version of the program in which the unsafe `strcpy()` call is replaced with `strncpy()`, explicitly limiting the number of bytes copied to the buffer size minus one, and manually appending a null terminator immediately after the copy. Also add a check that prints a warning whenever the supplied input is longer than the buffer can hold.
6. Compile this corrected program and run it with the same oversized input used earlier to confirm that the program no longer crashes and instead truncates the input safely while continuing to execute normally.
7. For the second part of the experiment, write a Python script that creates an in-memory SQLite database containing a users table with username and password columns, and insert two sample user records into it.
8. In the same script, write a login function that builds its SQL query by directly concatenating the supplied username and password strings into the `SELECT` statement, and run it once with a valid username and password to confirm that a normal login succeeds.
9. Run the same login function again, this time supplying a crafted username that contains a SQL comment sequence in place of a real username, and observe that the query returns a match and the login is reported as successful even though the correct password was never supplied, confirming that the authentication check has been bypassed.
10. Rewrite the login function so that the username and password are passed to the database driver as separate bound parameters using placeholders, instead of being concatenated into the query text, and save this as the secure version of the script.
11. Run the secure version with the same crafted username used earlier and confirm that the payload is now treated purely as literal text rather than as part of the SQL command, so that the login attempt correctly fails.

Source Code — Buffer Overflow (Vulnerable Version)

```
/* -----
File   : AnshumanDash_BufferOverflow_Vulnerable.c
Author : Anshuman Dash
Aim    : Demonstrate an unsafe C function that does not
         perform bounds checking while copying user input
         into a fixed size stack buffer.
----- */
#include <stdio.h>
#include <string.h>

void copyInput(char *input) {
    char buffer[16]; /* fixed size buffer, only 16 bytes */
    strcpy(buffer, input); /* no length check -> overflow possible */
    printf("Buffer content : %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input string>\n", argv[0]);
        return 1;
    }
    printf("Length of input received : %lu bytes\n", strlen(argv[1]));
    copyInput(argv[1]);
    printf("Program finished normally.\n");
    return 0;
}
```

Source Code — Buffer Overflow (Secure Version)

```
/* -----
File   : AnshumanDash_BufferOverflow_Secure.c
Author : Anshuman Dash
Aim    : Fix the buffer overflow by validating length and
         using a bounded copy function (strncpy) that can
         never write past the end of the destination buffer.
----- */
#include <stdio.h>
#include <string.h>

#define BUFFER_SIZE 16

void copyInputSecurely(const char *input) {
    char buffer[BUFFER_SIZE];

    /* Bounded copy: never writes more than sizeof(buffer)-1 bytes */
    strncpy(buffer, input, BUFFER_SIZE - 1);
    buffer[BUFFER_SIZE - 1] = '\0'; /* guarantee null termination */

    if (strlen(input) >= BUFFER_SIZE) {
        printf("Warning: input truncated to fit %d byte buffer safely.\n",
              BUFFER_SIZE - 1);
    }
    printf("Buffer content : %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input string>\n", argv[0]);
        return 1;
    }
    printf("Length of input received : %lu bytes\n", strlen(argv[1]));
    copyInputSecurely(argv[1]);
}
```

```

    printf("Program finished normally.\n");
    return 0;
}

```

Source Code — SQL Injection (Vulnerable Version)

```

"""
File   : AnshumanDash_SQLInjection_Vulnerable.py
Author : Anshuman Dash
Aim    : Demonstrate a login check that builds its SQL query by
         string concatenation, which lets an attacker inject
         SQL logic through the input fields.
"""

import sqlite3

def setup_db():
    conn = sqlite3.connect(":memory:")
    cur = conn.cursor()
    cur.execute("CREATE TABLE users (username TEXT, password TEXT)")
    cur.execute("INSERT INTO users VALUES ('admin', 'Sup3rSecret!')")
    cur.execute("INSERT INTO users VALUES ('anshuman', 'MyPass123')")
    conn.commit()
    return conn

def login_vulnerable(conn, username, password):
    cur = conn.cursor()
    # UNSAFE: user input concatenated directly into the query string
    query = "SELECT * FROM users WHERE username = '" + username + \
            "' AND password = '" + password + "'"
    print("Executed query :", query)
    cur.execute(query)
    return cur.fetchall()

if __name__ == "__main__":
    conn = setup_db()

    print("\n--- Attempt 1: legitimate login ---")
    result = login_vulnerable(conn, "admin", "Sup3rSecret!")
    print("Login result   :", "SUCCESS" if result else "FAILED")

    print("\n--- Attempt 2: SQL injection payload in username field ---")
    malicious_username = "admin' --"
    result = login_vulnerable(conn, malicious_username, "wrong_password")
    print("Login result   :", "SUCCESS (AUTH BYPASSED!)" if result else "FAILED")

```

Source Code — SQL Injection (Secure Version)

```

"""
File   : AnshumanDash_SQLInjection_Secure.py
Author : Anshuman Dash
Aim    : Fix the SQL injection flaw by using parameterized
         queries (placeholders) so user input is always treated
         as data, never as part of the SQL command.
"""

import sqlite3

def setup_db():
    conn = sqlite3.connect(":memory:")
    cur = conn.cursor()
    cur.execute("CREATE TABLE users (username TEXT, password TEXT)")
    cur.execute("INSERT INTO users VALUES ('admin', 'Sup3rSecret!')")
    cur.execute("INSERT INTO users VALUES ('anshuman', 'MyPass123')")
    conn.commit()
    return conn

```

```

def login_secure(conn, username, password):
    cur = conn.cursor()
    # SAFE: placeholders (?) keep user input separate from SQL logic
    query = "SELECT * FROM users WHERE username = ? AND password = ?"
    print("Executed query :", query, " | params:", (username, password))
    cur.execute(query, (username, password))
    return cur.fetchall()

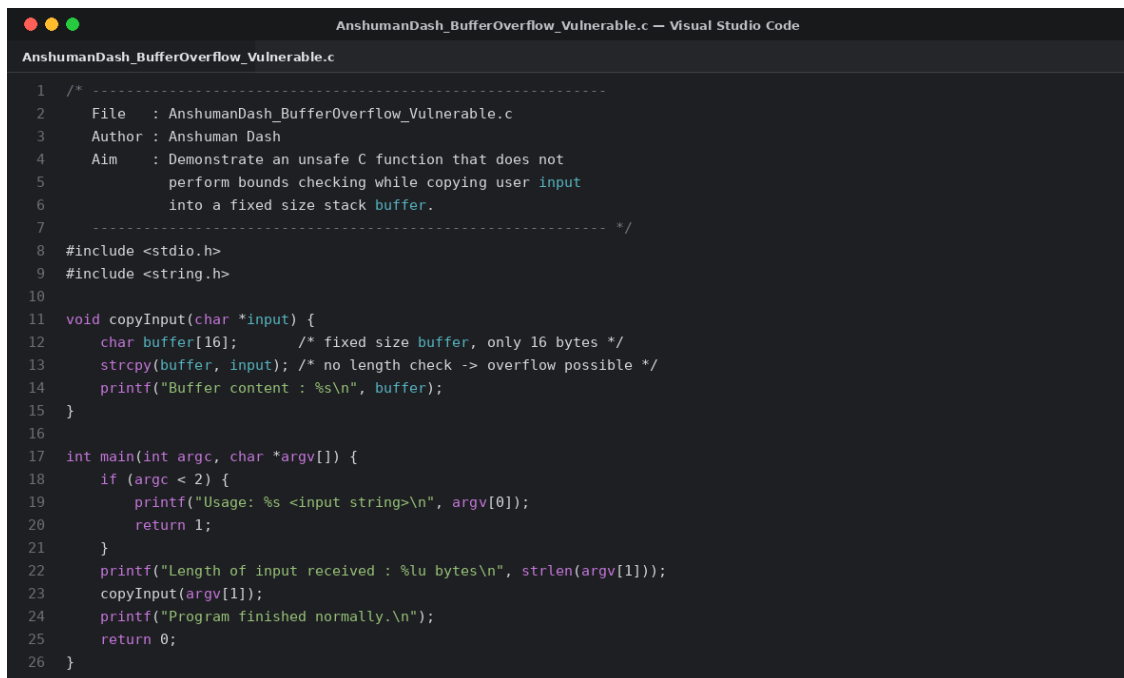
if __name__ == "__main__":
    conn = setup_db()

    print("\n--- Attempt 1: legitimate login ---")
    result = login_secure(conn, "admin", "Sup3rSecret!")
    print("Login result   :", "SUCCESS" if result else "FAILED")

    print("\n--- Attempt 2: same SQL injection payload as before ---")
    malicious_username = "admin' --"
    result = login_secure(conn, malicious_username, "wrong_password")
    print("Login result   :", "SUCCESS (AUTH BYPASSED!)" if result else "FAILED (payload treated as plain text)")

```

Output / Screenshots

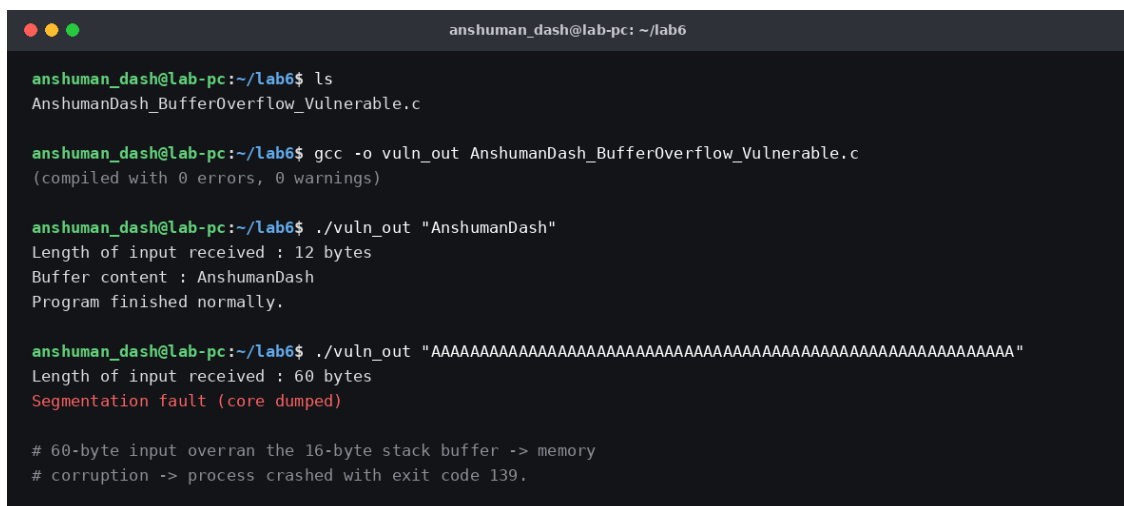


```

AnshumanDash_BufferOverflow_Vulnerable.c — Visual Studio Code
AnshumanDash_BufferOverflow_Vulnerable.c
1  /* -----
2  File   : AnshumanDash_BufferOverflow_Vulnerable.c
3  Author : Anshuman Dash
4  Aim    : Demonstrate an unsafe C function that does not
5            perform bounds checking while copying user input
6            into a fixed size stack buffer.
7  ----- */
8  #include <stdio.h>
9  #include <string.h>
10
11 void copyInput(char *input) {
12     char buffer[16]; /* fixed size buffer, only 16 bytes */
13     strcpy(buffer, input); /* no length check -> overflow possible */
14     printf("Buffer content : %s\n", buffer);
15 }
16
17 int main(int argc, char *argv[]) {
18     if (argc < 2) {
19         printf("Usage: %s <input string>\n", argv[0]);
20         return 1;
21     }
22     printf("Length of input received : %lu bytes\n", strlen(argv[1]));
23     copyInput(argv[1]);
24     printf("Program finished normally.\n");
25     return 0;
26 }

```

Fig. 1 — Editor view of the vulnerable buffer overflow program saved as *AnshumanDash_BufferOverflow_Vulnerable.c*



```

anshuman_dash@lab-pc: ~/lab6
anshuman_dash@lab-pc:~/lab6$ ls
AnshumanDash_BufferOverflow_Vulnerable.c

anshuman_dash@lab-pc:~/lab6$ gcc -o vuln_out AnshumanDash_BufferOverflow_Vulnerable.c
(compiled with 0 errors, 0 warnings)

anshuman_dash@lab-pc:~/lab6$ ./vuln_out "AnshumanDash"
Length of input received : 12 bytes
Buffer content : AnshumanDash
Program finished normally.

anshuman_dash@lab-pc:~/lab6$ ./vuln_out "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
Length of input received : 60 bytes
Segmentation fault (core dumped)

# 60-byte input overran the 16-byte stack buffer -> memory
# corruption -> process crashed with exit code 139.

```

Fig. 2 — Compiling and running the vulnerable program: a normal input works, but an oversized input causes a segmentation fault.

```
anshuman_dash@lab-pc: ~/lab6

anshuman_dash@lab-pc:~/lab6$ gcc -o secure_out AnshumanDash_BufferOverflow_Secure.c
(compiled with 0 errors, 0 warnings)

anshuman_dash@lab-pc:~/lab6$ ./secure_out "AnshumanDash"
Length of input received : 12 bytes
Buffer content : AnshumanDash
Program finished normally.

anshuman_dash@lab-pc:~/lab6$ ./secure_out "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
Length of input received : 60 bytes
Warning: input truncated to fit 15 byte buffer safely.
Buffer content : AAAAAAAAAAAAAA
Program finished normally.

# strncpy() + explicit null-termination stopped the write
# at the buffer boundary -> no crash, no memory corruption.
```

Fig. 3 — The secure version truncates the oversized input safely instead of crashing.

```
anshuman_dash@lab-pc: ~/lab6

anshuman_dash@lab-pc:~/lab6$ python3 AnshumanDash_SQLInjection_Vulnerable.py

--- Attempt 1: legitimate login ---
Executed query : SELECT * FROM users WHERE username = 'admin'
                  AND password = 'Sup3rSecret!'
Login result   : SUCCESS

--- Attempt 2: SQL injection payload in username field ---
Executed query : SELECT * FROM users WHERE username = 'admin' --'
                  AND password = 'wrong_password'
Login result   : SUCCESS (AUTH BYPASSED!)

# the -- sequence commented out the password check,
# so ANY password logs in as admin.
```

Fig. 4 — The vulnerable login script: the crafted username bypasses authentication without a valid password.

```
anshuman_dash@lab-pc: ~/lab6

anshuman_dash@lab-pc:~/lab6$ python3 AnshumanDash_SQLInjection_Secure.py

--- Attempt 1: legitimate login ---
Executed query : SELECT * FROM users WHERE username = ? AND
                  password = ? | params: ('admin', 'Sup3rSecret!')
Login result   : SUCCESS

--- Attempt 2: same SQL injection payload as before ---
Executed query : SELECT * FROM users WHERE username = ? AND
                  password = ? | params: ("admin' --", 'wrong_password')
Login result   : FAILED (payload treated as plain text)

# the placeholder (?) binds the payload as literal data, so
# it can never change the SQL command's structure.
```

Fig. 5 — The secure, parameterized login script correctly rejects the same injection payload.